



Wrocław
University
of Science
and Technology



Serverless Computing with AWS Lambda

Cloud Programming (W04IST-SI0826G)

Wojciech Thomas, PhD

Department of Applied Informatics
Faculty of Information and Communication Technology

Spring 2026



Why this matters

- An e-commerce site runs a flash sale — order confirmations arrive between 1 AM and 3 AM
- An ECS Fargate cluster must run 24/7 to handle those two hours of load: idle capacity costs money regardless
- Lambda wakes only when an event fires — you pay per invocation, not per idle hour
- Knowing when to switch compute models can reduce costs by 80 % or more on intermittent workloads



This week

After this class, you should be able to:

- **Explain** what serverless computing is and how it differs from container-based deployment (ECS)
- **Explain** how Lambda's stateless execution model affects application design choices
- **Distinguish** among the three invocation models and select the appropriate one for a use case
- **Distinguish** the Lambda execution role from a resource-based policy and explain the purpose of each
- **Compare** the cost model of Lambda against always-on compute for variable-traffic workloads



Agenda

- 1 Serverless Computing Fundamentals
- 2 Lambda Function Anatomy
- 3 Event Sources and Integrations
- 4 Security and Pricing



From servers to serverless

Abstraction	You manage	Cloud manages
Bare metal	Everything	Physical hardware
Virtual machines (EC2)	OS, runtime, app	Hardware, hypervisor
Containers (ECS Fargate)	App code, image	OS, runtime, cluster
Functions (Lambda)	App code only	Everything else

- Key shift: from “*how much capacity do I provision?*” to “*what should happen when this event fires?*”
- Serverless is not a replacement for containers – it is a different tool for a different problem shape



What is AWS Lambda

AWS Lambda

An **event-driven**, serverless compute service that runs your code in response to events – no servers to provision, patch, or manage.

- Unit of deployment: a *function* – single-purpose, stateless handler
- Managed runtimes: Node.js, Python, Java, .NET, Ruby
- Compiled languages (Go, Rust): OS-only custom runtimes (`provided.al2023`) – not managed runtimes
- Maximum execution duration: **15 minutes** – not suitable for long-running jobs (contrast with ECS)



Lambda execution lifecycle

- Three-phase lifecycle per invocation: **Init** → **Invoke** → **Shutdown**
- **Cold start**: Init phase runs when no warm environment is available
 - Download deployment package, initialize the runtime, run module-level initialization code
 - Adds 100 ms–2 s depending on runtime and package size
- **Warm start**: subsequent invocations reuse the existing execution environment – no Init overhead
- Environment reuse is *opportunistic* – never write code that *requires* a warm environment to be correct



Stateless design pattern

- Each invocation is independent – no in-memory state persists between calls
- Multiple execution environments run in parallel – global variables are *not shared* across invocations
- Ephemeral `/tmp` storage: up to 10 GB, writable per environment, not shared across concurrent invocations

All persistent state must live in an external service:

Storage requirement	Service
Structured records	DynamoDB
Files and objects	S3
Session cache	ElastiCache



Before we continue — predict

A function is deployed with a module-level counter:

```
1 counter = 0
2 def handler(event, context):
3     global counter
4     counter += 1
5     return {"invocations": counter}
```

The function is called 5 times in quick succession. What does the response look like?

- 1 Always {"invocations": 1} — each invocation starts fresh
- 2 Counts 1 to 5 reliably — module state is persistent
- 3 Unpredictable — depends on how Lambda assigns environments

3 — **Unpredictable**: invocations on the same warm environment increment the counter; a new cold-start environment restarts at 0. Module-level state should never be used as a shared counter or session store.



Agenda

- 1 Serverless Computing Fundamentals
- 2 Lambda Function Anatomy
- 3 Event Sources and Integrations
- 4 Security and Pricing



Function handler

- Handler naming convention: `module.function_name` — tells Lambda the entry point
- Python handler receives two objects: `event` (trigger payload) and `context` (runtime metadata)

```
1 import json
2 def handler(event, context):
3     name = event.get("name", "world")
4     return {
5         "statusCode": 200,
6         "body": json.dumps({"message": f"Hello, {name}!"})
7     }
```



Deploying the function handler

Deploy the function

```
1 aws lambda create-function --function-name hello --runtime
  ↪ python3.12 --handler lambda_function.handler ...`
```

Invoke the function

```
1 aws lambda invoke --function-name hello --payload
  ↪ '{"name": "student"}' out.json
```



Event and context objects

- **event**: the trigger payload – JSON structure varies by event source

```
1 {
2   "httpMethod": "GET",
3   "path": "/hello",
4   "queryStringParameters": {"name": "student"},
5   "body": None
6 }
```

- **context**: runtime metadata injected by Lambda – `function_name`, `memory_limit_in_mb`, `aws_request_id`
- `context.get_remaining_time_in_millis()` – milliseconds until the hard timeout kills the invocation
- Use remaining time to exit gracefully rather than being terminated mid-operation



Function configuration

Parameter	Range	Key point
Memory	128 MB – 10 240 MB	CPU scales proportionally
Timeout	1 s – 900 s	Hard limit – Lambda kills at T+timeout
Environment vars	up to 4 KB total	Injected at runtime

- Configure with CLI:

```
1 aws lambda update-function-configuration --function-name
   ↪ hello --memory-size 512 --timeout 30
```

- **Never hard-code credentials** – use environment variables or AWS Secrets Manager
- Timeout is not a soft suggestion – plan for graceful degradation



Deployment options

Method	Size limit	Best for
ZIP (direct upload)	50 MB	Small functions, few dependencies
ZIP via S3	250 MB unzipped	Larger packages (e.g., with data science libs)
Container image	10 GB	Full control; existing Docker workflows from ECS

- **Lambda Layers**: shared libraries and runtimes reused across functions – avoids duplicating dependencies in every ZIP
- Container image deployment uses the same OCI format as ECS – existing Dockerfiles can be adapted with minimal changes



Admin challenge

Scenario

A Lambda function that resizes uploaded images **times out** after 3 seconds. Memory is set to 128 MB.

A colleague proposes: *“Just increase the timeout to 30 seconds.”*

What should you check first, and why?

- **Check memory first** – image resizing is CPU-bound; at 128 MB Lambda receives proportionally little CPU time
- Increasing the timeout only extends the ceiling – it does not speed up the work
- Increase memory to 512 MB or 1 GB; the function may complete in under 1 second
- *Important caveat*: this applies to **CPU-bound** workloads only – for I/O-bound functions waiting on an external API, more memory does not reduce wall-clock time and will simply increase cost



Agenda

- 1 Serverless Computing Fundamentals
- 2 Lambda Function Anatomy
- 3 Event Sources and Integrations
- 4 Security and Pricing



Invocation models

- **Synchronous** (RequestResponse): caller blocks waiting for the result
 - Errors returned directly to the caller; caller handles retries
 - Used by: API Gateway, Application Load Balancer, SDK/CLI direct calls
- **Asynchronous** (Event): Lambda returns HTTP 202 immediately; event queued internally
 - Caller never receives the function result; Lambda retries on failure (up to 2x)
 - Used by: S3 events, SNS notifications, EventBridge rules
- **Event source mapping (polling)**: Lambda polls the source; records batched and delivered
 - Used by: SQS queues, DynamoDB Streams, Kinesis Data Streams



Choosing the right invocation model

Use case	Model	Explanation
HTTP API endpoint	Synchronous	Client is waiting; needs a response or an error
File uploaded to S3	Asynchronous	S3 pushes the event; no caller waits for a result
Process a message queue	Event source mapping	Ordered, at-least-once delivery; Lambda controls the read rate
Scheduled nightly job	Asynchronous (EventBridge)	Timer fires the event; no response needed

- Key design consequence: synchronous callers receive errors directly; asynchronous callers do *not* — error handling and dead-letter queue strategies differ fundamentally between the two



Lambda and API Gateway

- Request flow: client → API Gateway → Lambda (synchronous invoke) → response to client
- API Gateway passes a structured **proxy event** to Lambda:

```
1 {  
2   "httpMethod": "POST",  
3   "path": "/orders",  
4   "headers": {"Content-Type": "application/json"},  
5   "body": [{"item": "book"}]  
6 }
```

- **Proxy integration** (default and recommended): API Gateway forwards the full HTTP request as-is
- Lambda must return `statusCode`, `headers`, and `body` — API Gateway delivers it to the client
- Custom integration: API Gateway transforms the request/response — more complex, rarely needed



Discuss with your neighbor (2 min)

Three scenarios — which invocation model fits each, and why?

- 1 An e-commerce checkout endpoint that must return an order ID to the waiting browser
 - 2 A new product image uploaded to S3 that needs a thumbnail generated
 - 3 A nightly report that aggregates DynamoDB records — the job runs for 20 minutes
-
- 1 **Synchronous** — the browser is waiting for the confirmation
 - 2 **Asynchronous** — S3 pushes the event; no client waits for the thumbnail
 - 3 **Neither** — 20 min exceeds Lambda's 15-min hard timeout; use ECS or AWS Batch instead



Agenda

- 1 Serverless Computing Fundamentals
- 2 Lambda Function Anatomy
- 3 Event Sources and Integrations
- 4 Security and Pricing



Lambda execution role

- **Execution role**: IAM role assumed by the Lambda service when your function runs
- Defines what the function is *allowed to do* — which AWS API calls it can make
- AWS-managed starting point: `AWSLambdaBasicExecutionRole` — grants CloudWatch Logs write access only
- Common mistake: attaching `AdministratorAccess` — grants full account control from any event trigger



Sample execution role policy

```
1 {  
2   "Effect": "Allow",  
3   "Action": ["s3:GetObject"],  
4   "Resource": "arn:aws:s3:::my-bucket/*"  
5 }
```

- Principle of least privilege: scope to the exact bucket, table, or secret the function actually needs



Resource-based policy (function policy)

- Attached to the Lambda function itself – *who can invoke* the function
- Required when the caller is *another AWS service* (S3, API Gateway, EventBridge)



Two directions of trust

IAM construct	Controls	Direction
Execution role	What Lambda <i>can do</i>	Outbound: Lambda → AWS services
Function policy	Who <i>can call</i> Lambda	Inbound: AWS service → Lambda

- Both must be correct for an integration to work: the calling service must be allowed to invoke Lambda, *and* Lambda must be allowed to call the downstream service



Before we continue – predict

Scenario

API Gateway → Lambda → S3 is configured and deployed. After deployment:

- API Gateway returns **403 Forbidden**
- CloudWatch shows **no Lambda log entries** – the function was never invoked

Which IAM construct is missing, and why?

- The **function policy** is missing – API Gateway does not have permission to invoke the Lambda function
- The execution role is irrelevant here: the function never ran
- Fix: add a resource-based policy statement granting `lambda:InvokeFunction` to the API Gateway service principal



Lambda in a VPC

- By default, Lambda runs outside your VPC – can reach the public internet, not private resources
- Attach to a VPC to reach: RDS instances, ElastiCache clusters, internal APIs
- Configuration: select subnets and a security group – same model as ECS task networking

Trade-off	Detail
Subnet IP consumption	One ENI per execution environment – plan IP space carefully
Outbound internet	NAT Gateway required – adds ~\$0.045/GB data charge and ~\$32/month fixed
Security groups	Same inbound/outbound rules as any VPC resource

- The old “VPC causes severe cold start latency” is no longer true – AWS Hyperplane resolved this around 2020 by pre-creating ENIs at function update time, not at invocation time



Lambda pricing model

- **Requests:** first 1 million free per month; \$0.20 per million thereafter
- **Duration:** 1 ms granularity; rate = memory (GB) × execution time (s) × \$0.0000166667 per GB-s
- Memory optimization for **CPU-bound** functions: more memory → faster execution → lower duration charge
 - *Does not apply* to I/O-bound functions waiting on external calls — external latency is unchanged

Workload	Lambda	ECS Fargate (0.25 vCPU / 0.5 GB, 24/7)
1 M × 500 ms × 512 MB Spiky (nights + weekends only)	≈ \$4.17 Scales to zero	≈ \$10.80/month always-on Same cost regardless of traffic



In the lab

You will:

- Write a Python Lambda handler
- Configure an API Gateway proxy integration trigger via the AWS CLI
- Test the endpoint end-to-end and observe cold start vs. warm invocation latency in CloudWatch Logs
- Estimate and compare the monthly cost of this workload vs. an equivalent always-on ECS Fargate task



References

- [1] Amazon Web Services, “AWS lambda developer guide,” 2026. <https://docs.aws.amazon.com/lambda/latest/dg/> (accessed May 26, 2026).
- [2] Amazon Web Services, “AWS lambda pricing,” 2026. <https://aws.amazon.com/lambda/pricing/> (accessed May 26, 2026).
- [3] J. Chapin and M. Roberts, *Programming AWS lambda: Build and deploy serverless applications with java*. O'Reilly Media, 2020.